

OOP AND REPRESENTATION Meta

COMPUTER SCIENCE MENTORS 61A

March 9 – March 13, 2026

Example Timeline

- OOP Basics [6 min]

Go over how to use dot notation to lookup attributes of classes and objects, the order of evaluation.

Refresh on inheritance, what does the syntax look like, etc.

- 1. Wicked [7 min]
- 2. DLL [12 min]
- 3. PingPong [8 min]
- Representation [4 min]

Next question goes over `__repr__` and `__str__` in more detail but feel free to briefly introduce things via mini lecture as well.

- 4. Reprtilia [10 min]
- 5. Photosynthesis [10 min]

1. What would Python display in the land of Oz?

```
class Elphaba(object):  
    magic = 'Defying Gravity'  
  
    def __init__(self, magic):  
        self.magic = magic  
  
    def spell(self):  
        return type(self).magic + ' and ' + self.magic  
  
class Glinda(Elphaba):  
    magic = 'Popular'  
  
    def __init__(self, magic):  
        Elphaba.__init__(self, 'good ' + magic)
```

```
elphaba = Elphaba('Wicked')
```

- (a) >>> Elphaba.magic
 'Defying Gravity'
(b) >>> elphaba.magic
 'Wicked'
(c) >>> elphaba.spell()
 'Defying Gravity and Wicked'
(d) >>> Elphaba.spell()
 Error
(e) >>> Elphaba.spell(elphaba)
 'Defying Gravity and Wicked'
(f) >>> glinda = Glinda('Power')
 >>> Glinda.magic
 'Popular'
(g) >>> glinda.magic
 'good Power'
(h) >>> glinda.spell()
 'Popular and good Power'

Suggested Time: 7 min; Difficulty: Easy

- What is the difference between Elphaba and elphaba in this problem? Which one is considered the class vs the object and how is attribute access affected?

- What does this line mean: `type(self).magic`? Think about what `self` is in that function
- Explain how overriding works when it comes to inheritance through examples.

2. The `DLList` class is a spin off of the normal `Link` class we learned in class; each `DLList` link has a `prev` attribute that keeps track of the previous link and a `next` attribute that keeps track of the next link. Fill in the following methods for the `DLList` class.

```
(a) class DLList:
    """
    >>> lst = DLList(6, DLList(1))
    >>> lst.value
    6
    >>> lst.next.value
    1
    >>> lst.prev.value
    AttributeError: 'NoneType' object has no attribute 'value'
    """
    empty = None
    def __init__(self, value, next=empty, prev=empty):
        _____
        _____
        _____
```

```
def __init__(self, value, next=empty, prev=empty):
    self.value = value
    self.next = next
    self.prev = prev
```

```
(b) def add_last(self, value):
    """
    >>> lst = DLList(6)
    >>> lst.add_last(1)
    >>> lst.value
    6
    >>> lst.next.value
    1
    >>> lst.next.prev.value
    6
    """
    pointer = self
    while _____:
        _____
    _____ = DLList(_____)
```

```

def add_last(self, value):
    pointer = self
    while pointer.next != DLLList.empty:
        pointer = pointer.next
    pointer.next = DLLList(value, DLLList.empty, pointer)

```

(c) **def** add_first(self, value):

```
    """
```

```
    >>> lst = DLLList('A')
```

```
    >>> lst.add_first(1)
```

```
    >>> lst.value
```

```
    1
```

```
    >>> lst.next.value
```

```
    'A'
```

```
    >>> lst.next.prev.value
```

```
    1
```

```
    >>> lst.add_first(6)
```

```
    >>> lst.value
```

```
    6
```

```
    >>> lst.next.next.prev.value
```

```
    1
```

```
    """
```

```
    old_first = DLLList(_____)
```

```
    _____ = _____
```

```
    _____ = _____
```

```
    if _____:
```

```
        _____
```

```

def add_first(self, value):

```

```
    old_first = DLLList(self.value, self.next, self)
```

```
    self.value = value
```

```
    self.next = old_first
```

```
    if old_first.next != DLLList.empty:
```

```
        old_first.next.prev = old_first
```

Teaching Tips

- As always, remind students that they do not need to go in order when completing OOP questions. It's usually easiest to start out with what is given in the `__init__` method first and then seeing what each of the other methods need and how to get those values.
- It's recommended that you draw out several examples of a doubly linked list as having twice as many pointers can get confusing. Luckily your students will be able to draw on their knowledge of working with a one-ended LinkedList so it shouldn't be too bad.
- Tell your students to draw out the lists just like you so they can visualize which arrows need to be broken or created in order to complete each method. Encourage them to think of different edge cases, such as having a one-element list, or having a circular list, to ensure that their method is comprehensive.

Suggested Time: 12 min; Easy/Medium

- More difficult question that requires students to also think about the design of a doubly linked list. Feel free to help them with any logic/design questions they have.

3. Let's use OOP to help us implement our good friend, the ping-pong sequence!

As a reminder, the ping-pong sequence counts up starting from 1 and is always either counting up or counting down.

At element k , the direction switches if k is a multiple of 7 or contains the digit 7.

The first 30 elements of the ping-pong sequence are listed below, with direction swaps marked using brackets at the 7th, 14th, 17th, 21st, 27th, and 28th elements:

1 2 3 4 5 6 [7] 6 5 4 3 2 1 [0] 1 2 [3] 2 1 0 [-1] 0 1 2 3 4
[5] [4] 5 6

Assume you have a function `has_seven(k)` that returns True if k contains the digit 7.

```
>>> tracker1 = PingPongTracker()
>>> tracker2 = PingPongTracker()
>>> tracker1.next()
1
>>> tracker1.next()
2
>>> tracker2.next()
1
```

```
class PingPongTracker:
    def __init__(self):
```

```
        def next(self):
```

```
class PingPongTracker:
    def __init__(self):
        self.current = 0
        self.index = 1
        self.add = True

    def next(self):
        if self.add:
            self.current += 1
        else:
            self.current -= 1
        if has_seven(self.index) or self.index % 7 == 0:
            self.add = not self.add
        self.index += 1
        return self.current
```

Teaching Tips

- Emphasize the fact that the important part of such sequence problems are *keeping track of state at a given time step*. With OOP, this state is inherently saved as object attributes.
- Make sure the difference between `self.current` and `self.index` is clear: `index` always increases by 1 each step, while `current` is the actual pingpong sequence number we want.
- Remember that the **index** denotes the the progress we've made along the pingpong sequence- `current` is just the current number of our sequence we happen to be on.
- Students may have seen a version of pingpong that uses -1 and 1 as a direction variable instead of a boolean "add" variable. Make sure to clarify how the add variable operates here, and how it differs from the -1/1 version.

Suggested Time: 10 min; Medium

- Less scaffolding than previous questions and feel free to spend more time on this problem. Also spend some time explaining how the pingpong OOP representation works and what students have to do.

2 Representation

1. The `Reprtilia` class stores the common and scientific names of various reptiles.

Brief introduction to `__repr__`

The `__repr__` magic method of objects returns the "official" string representation of an object. You can invoke it directly by calling `repr(<some object>)`. However, `__repr__` doesn't always return something that is easily readable, that is what `__str__` is for. Rather, `__repr__` ensures that all information about the object is present in the representation. When you ask Python to represent an object in the Python interpreter, it will automatically call `repr` on that object and then print out the string that `repr` returns.

```
"""
>>> python = Reprtilia("python", "pythonidae")
>>> iguana = Reprtilia("iguana", "iguana")
>>> python
Reprtilia('python', 'pythonidae')
>>> f'Did you know that {python}?'
'Did you know that a python is a "pythonidae"?'
"""

class Reprtilia:
    def __init__(self, name, scientific):
        self.name = name
        self.scientific = scientific
```

2. How do we define the `__repr__` method?

```
def __repr__(self):  
  
    return f"Reprtilia({repr(self.name)}, {repr(self.scientific)})"
```

3. How do we define the `__str__` method?

```
def __str__(self):  
  
    return f"a {self.name} is a \"{self.scientific}\""
```


4. Your friend, Julian, decides to create a new object class based off of `Reprtilia` called `ReprtiliaNoises`, and as such, decides to overhaul your `__repr__` and `__str__`! How would the following be implemented following the doctests?

```
"""
>>> python = ReprtiliaNoises("python", "pythonidae", "hiss")
>>> python
ReprtiliaNoises('python', 'pythonidae', 'hiss')
>>> f"Today's reptile of the day is a {python}!"
"Today's reptile of the day is a python (species: 'pythonidae',
    noise: 'hiss')!"
"""
```

5. How does Julian redefine the `__init__` method?

```
class ReprtiliaNoises(Reprtilia):
    def __init__(self, name, scientific, noise):

        super().__init__(name, scientific)
        self.noise = noise
```

6. How does Julian redefine the `__repr__` and `__str__` method?

```
def __repr__(self):

    return f"ReprtiliaNoises({repr(self.name)},
                             {repr(self.scientific)}, {repr(self.noise)})"
```

7. How does Julian redefine the `__str__` method?

```
def __str__(self):

    return f"{self.name} (species: '{self.scientific}',
            noise: '{self.noise}')
```

Suggested Time: 10 min; Difficulty: Easy

- Introduces the distinction between the `__repr__` and `__str__` methods in Python and how it affects how objects are displayed.
- Explain how when evaluating an object in the interpreter, Python will call `repr(obj)`
- If an object is placed in an f-string or `print` is called on it, Python will call `str(obj)`

8. Implement the classes so the following code runs.

```
'''
>>> p = Plant()
>>> p.height
1
>>> p.materials
[]
>>> p.absorb()
>>> p.materials
[|Sugar|]
>>> Sugar.sugars_created
1
>>> p.leaf.sugars_used
0
>>> p.grow()
>>> p.materials
[]
>>> p.height
2
>>> p.leaf.sugars_used
1
'''
```

```
class Plant:
    def __init__(self):
        self.leaf = Leaf(self)
        self.materials = []
        self.height = 1

    def absorb(self):
        self.leaf.absorb()

    def grow(self):
        for sugar in self.materials:
            sugar.activate()
            self.height += 1

class Leaf:
    def __init__(self, plant): # plant is a Plant instance
        self.alive = True
        self.sugars_used = 0
        self.plant = plant

    def absorb(self):
        if self.alive:
            self.plant.materials.append(Sugar(self, self.plant))

    def __repr__(self):
        return '|Leaf|'
```

```

class Sugar:
    sugars_created = 0

    def __init__(self, leaf, plant):
        self.leaf = leaf
        self.plant = plant
        Sugar.sugars_created += 1

    def activate(self):
        self.leaf.sugars_used += 1
        self.plant.materials.remove(self)

    def __repr__(self):
        return '|Sugar|'

```

Suggested Time: 10 min; Difficulty: Medium

- This question revolves around how objects interact with each other and tracking a shared state
- Explain why we save a leaf object as an attribute of Plant and why we can do Leaf(self)
- As always spend some time going over the role of a Plant vs Leaf vs Sugar and feel free to answer as many questions relating to That